

Prompt: app de campo bananera (demo)

Oct 4, 2026 · @hemi

Cómo usar este prompt

Copie desde la sección «Rol y objetivo» hasta el final y péguelo en un agente de código (por ejemplo, Claude Code) dentro de una carpeta vacía.

- **Adjunte también** la *Guía de marca para informes* de Inversiones Kalo (PDF) y el archivo del logo (`kalo-logo.png`), y colóquelos en `docs/marca/` del proyecto.
- **Datos pendientes de la finca:** colores reales del ciclo de cintas, polígonos de los lotes y lista de labores. El agente debe dejarlos como configuración y no inventarlos.
- Puede exportar este documento a Markdown y guardarlo en el repositorio como `docs/PROMPT.md`.

Rol y objetivo

Actúe como arquitecto y desarrollador full stack senior. Construya un **demo funcional** de una plataforma de recolección de datos de campo para la bananera de Inversiones Kalo, en Guatemala.

- Cree el monorepo completo: carpetas, configuración, código, base de datos, datos de demostración y documentación para ejecutarlo.
- El demo debe poder mostrarse a gerencia: una app móvil que funcione **sin señal** y sincronice al recuperarla, y un panel web que muestre los datos recolectados.
- Priorice que todo lo incluido funcione de punta a punta antes que agregar más módulos.
- Deje la arquitectura lista para crecer sin reescribir: nuevos módulos, formularios, fincas y roles.
- Escriba el código, los comentarios y la documentación en español; nombres de variables y tablas en inglés o español, pero de forma consistente.
- Nunca invente cifras de producción reales. Los datos de prueba deben marcarse como «DEMO».

Contexto del negocio y glosario

La bananera produce banana Cavendish de exportación. Los trabajadores de campo recorren lotes sin cobertura celular y hoy registran en papel. El sistema debe reemplazar ese papel.

Término	Significado
Finca	Unidad productiva; contiene lotes
Lote	Área delimitada (polígono) de la finca; es el centro de todos los registros
Enfunde	Colocar bolsa y cinta de color al racimo nuevo cada semana
Cinta	Color asignado a cada semana; indica la edad del racimo (cosecha a 11-13 semanas)
Preaviso de sigatoka	Lectura semanal de la enfermedad: hoja más joven enferma, estado de evolución, severidad
Población	Plantas productivas por hectárea (referencia: 1.600)
Retorno	Racimos por planta al año (referencia: 1,80-1,90)
Recobro	Proporción de racimos enfundados que se cosechan (referencia: 0,98)
Factor	Cajas por racimo; varía por semana del año (referencia: 1,15-1,50)
Caporal	Encargado de una cuadrilla de trabajadores
Destajo	Pago por tarea realizada (racimos, plantas, metros)

Ecuación productiva, que el panel web debe calcular por lote:

$$\text{Población} \times \text{Retorno} \times \text{Recobro} \times \text{Factor} = \text{cajas/ha/año}$$

Ejemplo de referencia: $1.600 \times 1,85 \times 0,98 \times 1,37 = 3.974$ cajas/ha/año.

Factor por semana del año (puntos conocidos; interpolar linealmente entre ellos y dejarlo editable): semana 1 = 1,40; semana 13 = 1,15; semanas 26-27 = 1,50; semanas 39-40 = 1,50; semana 52 = 1,40.

Alcance del demo

Incluya estos módulos funcionando de punta a punta. Lo marcado como «preparar» lleva solo estructura, rutas y pantallas vacías con aviso «Próximamente».

App móvil (trabajadores de campo)

1. **Inicio de sesión rápido:** usuario y PIN de 4 dígitos, o escaneo de gafete QR. Debe funcionar sin señal una vez que el dispositivo se configuró.
2. **Inicio del día:** muestra finca, lote actual detectado por GPS, tareas asignadas y estado de sincronización.
3. **Plagas y enfermedades:** muestreo por lote (plaga, incidencia, severidad, foto, GPS, nota de voz opcional) y formulario de preaviso de sigatoka.
4. **Trampas de picudo:** escanear QR de la trampa y registrar la cantidad capturada.
5. **Alerta de Fusarium:** botón visible en el inicio; foto, síntomas marcables y GPS; queda con estado «Sospecha».
6. **Enfunde:** racimos enfundados por lote, con el color de cinta de la semana seleccionado automáticamente.
7. **Cosecha:** racimos cosechados y perdidos por lote y color de cinta.
8. **Labores y asistencia:** el caporal registra asistencia y tareas por trabajador o cuadrilla (tipo de labor, lote, cantidad).
9. **Rutas GPS:** grabar el recorrido mientras una tarea está activa y ver la cobertura del lote.
10. **Sincronización:** pantalla con registros pendientes, fotos en cola, último envío y botón «Sincronizar ahora».
11. **Preparar:** conteo de cintas con cámara, detector de sigatoka por foto, aplicaciones de agroquímicos, bioseguridad, seguridad ocupacional, botón de emergencia.

Panel web (oficina)

1. Mapa de la finca con lotes coloreados por último estado de plagas y capa de rutas recorridas.
2. Tablas de registros por módulo con filtros (fecha, lote, usuario) y exportación a Excel y CSV.
3. Calculadora de la ecuación productiva por lote y pronóstico semanal de cajas (racimos por color × recobro × factor de la semana).
4. Bandeja de validación del supervisor y bandeja de alertas de Fusarium.
5. Administración: usuarios, roles y permisos, lotes, catálogo de plagas, labores, calendario de semanas y colores de cinta, parámetros.
6. Panel de dispositivos: último sincronizado, versión de la app y registros pendientes reportados.

7. **Preparar:** expedientes y nómina, reportería, cumplimiento, imágenes satelitales.

Stack tecnológico y arquitectura

Use TypeScript en todo el proyecto, salvo el servicio de visión, que se prepara en Python.

Capa	Tecnología
Monorepo	pnpm workspaces + Turborepo
App móvil	Expo (React Native) + Expo Router, TypeScript, Android como prioridad
Base local móvil	WatermelonDB (SQLite) como fuente principal de datos
Mapas móvil	MapLibre (<code>@maplibre/maplibre-react-native</code>) con descarga de mapas offline
GPS	<code>expo-location</code> + <code>expo-task-manager</code> para rastreo en segundo plano
Cámara y QR	<code>expo-camera</code> (incluye lectura de códigos QR)
Panel web	Next.js (App Router) + Tailwind CSS + MapLibre GL JS + TanStack Table + Recharts
API	Node.js + Fastify, validación con Zod, documentación OpenAPI en <code>/docs</code>
Base de datos	PostgreSQL 16 + PostGIS, ORM Drizzle con migraciones
Archivos	MinIO (compatible con S3) con URLs prefirmadas
Autenticación	JWT con renovación; PIN local cifrado para uso offline
Infraestructura	Docker Compose para Postgres, MinIO y API
Visión (preparar)	Python + FastAPI, carpeta lista para modelos YOLO/TFLite
Errores	Sentry configurado por variable de entorno (opcional en el demo)

Arquitectura: la app escribe siempre en su base local; un motor de sincronización envía y recibe cambios contra la API (`/v1/sync/pull` y `/v1/sync/push`). La API guarda en PostgreSQL y los archivos en MinIO. El panel web consume la misma API. Los tipos,

esquemas Zod y constantes viven en un paquete compartido para que móvil, web y API usen las mismas reglas.

Estructura de carpetas

Cree exactamente esta estructura base; agregue subcarpetas cuando lo necesite, manteniendo la organización por módulo.

```

kalo-campo/
├── apps/
│   ├── mobile/
│   │   ├── app/
│   │   │   ├── (auth)/
│   │   │   └── (tabs)/
│   └── Perfil
│       ├── modulos/
│       └── src/
│           ├── db/
│           ├── sync/
│           ├── gps/
│           └── modulos/
│   └── enfunde...
│       ├── formularios/
│       ├── permisos/
│       ├── componentes/
│       ├── i18n/
│       └── utils/
│   ├── web/
│   └── app/
├── admin...
│   ├── src/
│   ├── api/
│   └── src/
│       ├── modulos/
│       └── sync/
├── conflictos
│   ├── auth/
│   ├── archivos/
│   └── db/
├── vision/
├── packages/
│   ├── shared/
│   └── ui-tokens/
├── tipografía)
└── config/
# Expo + React Native
# Rutas (Expo Router)
# Login, PIN, configuración del dispositivo
# Inicio, Registrar, Mapa, Sincronizar,
# Pantallas por módulo
# WatermelonDB: schema, modelos, migraciones
# Motor de sincronización y cola de archivos
# Rastreo, geocercas, cálculo de lote actual
# Un folder por módulo (plagas, trampas,
# Motor de formularios dinámicos
# Roles, permisos y feature flags
# UI con la marca Kalo
# es (base) + carpeta para otros idiomas
# Next.js (panel de oficina)
# Rutas: mapa, registros, pronóstico,
# componentes, hooks, cliente de API
# Fastify
# Rutas y servicios por módulo
# Endpoints pull/push y resolución de
# URLs prefiradas de MinIO
# Drizzle: esquema, migraciones, seed
# Python + FastAPI (preparar)
# Tipos, esquemas Zod, constantes, cálculos
# Tokens de diseño Kalo (colores,
# ESLint, TypeScript, Prettier compartidos

```

```
├─ infra/
│   └─ docker-compose.yml          # Postgres+PostGIS, MinIO, API
├─ docs/
│   └─ marca/                      # Guía de marca PDF y logo
│       ├── arquitectura.md
│       ├── sincronizacion.md
│       └─ modulos.md
├─ .env.example
└─ README.md
```

Modelo de datos

Todas las tablas sincronizables llevan: `id` (UUID generado en el dispositivo), `created_at`, `updated_at`, `server_updated_at`, `deleted_at` (borrado lógico), `device_id`, `created_by` y `finca_id`. Las tablas con ubicación usan tipos PostGIS.

Grupo	Tablas
Organización	<code>empresas</code> , <code>fincas</code> , <code>lotes</code> (polígono, hectáreas, población), <code>dispositivos</code>
Usuarios	<code>usuarios</code> , <code>roles</code> , <code>permisos</code> , <code>rol_permisos</code> , <code>cuadrillas</code> , <code>cuadrilla_miembros</code>
Calendario	<code>semanas</code> (año, número, color de cinta, factor), <code>colores_cinta</code>
Sanidad	<code>plagas</code> (umbral de alerta), <code>muestreos</code> , <code>preaviso_sigatoka</code> , <code>trampas</code> (código QR, punto), <code>lecturas_trampa</code> , <code>alertas_fusarium</code> (estado)
Producción	<code>enfunde</code> , <code>cosecha</code> (racimos cosechados y perdidos), <code>conteos_cinta</code> , <code>pronosticos</code>
Personas	<code>trabajadores</code> (DPI único), <code>asistencia</code> , <code>tipos_labor</code> (unidad y tarifa), <code>labores</code>
GPS	<code>rutas</code> (usuario, tarea, inicio, fin), <code>puntos_ruta</code> (punto, precisión, hora del GPS), <code>cobertura_lote</code>
Tareas	<code>ordenes_trabajo</code> (asignado a, lote, estado, fecha)
Archivos	<code>archivos</code> (tipo, tamaño, estado de subida, registro al que pertenece)

Grupo	Tablas
Plataforma	modulos , definiciones_formulario (JSON versionado), feature_flags , parametros , bitacora

Reglas:

- El **lote** se relaciona con casi todo; la **semana** une enfunde, conteo, cosecha y pronóstico.
- Los registros críticos (cosecha, labores, alertas) tienen **estado_validacion**: pendiente, validado o rechazado.
- Ningún dato se borra físicamente desde la app; solo con borrado lógico.

Funcionamiento offline y sincronización

La señal en campo es muy mala: la app nunca debe esperar al servidor para guardar, mostrar o validar un registro.

1. **Offline primero:** toda escritura va a WatermelonDB. La interfaz lee solo de la base local.
2. **Protocolo:** implemente el protocolo de sincronización de WatermelonDB. **pull** devuelve cambios desde **last_pulled_at** (creados, actualizados, borrados); **push** recibe los cambios locales en lote y responde por registro.
3. **Varios dispositivos:** IDs UUID generados en el celular, **device_id** en cada registro y sincronización por cambios (delta), nunca la base completa.
4. **Conflictos:** por defecto gana el cambio más reciente según **updated_at** , campo por campo. En tablas críticas, el conflicto se guarda en **bitacora** y aparece en la bandeja del supervisor.
5. **Datos por dispositivo:** cada celular descarga solo su finca y los catálogos que su rol necesita.
6. **Cola de archivos aparte:** fotos comprimidas (máximo ~1.600 px, calidad 0,7) y notas de voz suben después de los datos, por URL prefirrada, con reintentos. Opción «Subir archivos solo con WiFi».
7. **Disparadores:** al abrir la app, al recuperar red (NetInfo), cada 15 minutos con red y con el botón «Sincronizar ahora». Reintentos con espera exponencial.
8. **Visibilidad:** indicador permanente en la barra (sincronizado, pendiente, error) con el número de registros y archivos en cola.
9. **Hora confiable:** guardar la hora del dispositivo y la hora del GPS en cada registro.

10. **Respaldo:** opción para exportar la base local cifrada a un archivo si un celular nunca logra sincronizar.
11. **Seguridad:** base local cifrada (SQLCipher o equivalente), sesión con caducidad y borrado remoto del dispositivo desde el panel.
12. **Pruebas:** incluya pruebas del motor de sincronización con dos dispositivos simulados editando el mismo registro sin conexión.

Documente el diseño completo en `docs/sincronizacion.md`.

GPS, rutas y cobertura por lote

El GPS sirve para saber dónde se tomó cada registro y qué partes de cada hectárea fueron revisadas.

- **Lote automático:** con los polígonos descargados, la app detecta en qué lote está el trabajador (punto dentro de polígono) y lo propone en cada formulario.
- **Rastreo por tarea:** se graba la ruta solo mientras haya una tarea o recorrido activo, nunca todo el día. Intervalo configurable (por defecto cada 10 s o 10 m) y modo ahorro de batería.
- **Segundo plano:** el rastreo continúa con la pantalla apagada, con notificación visible de que está activo.
- **Calidad:** descartar puntos con precisión peor a 30 m y simplificar la ruta antes de sincronizar.
- **Cobertura:** en el servidor, dividir cada lote en una cuadrícula (por defecto celdas de 20 × 20 m, configurable) y marcar las celdas recorridas con PostGIS. El panel muestra el porcentaje cubierto por lote y semana; la app muestra el mapa de cobertura.
- **Mapas offline:** descargar el mapa base y los polígonos de la finca al configurar el dispositivo.
- **Consentimiento:** pantalla de aviso y aceptación del rastreo la primera vez, guardada en la base.

Roles y permisos

Los permisos se guardan en la base y se editan desde el panel; nunca se fijan en el código. Cada permiso tiene la forma `modulo:accion` (por ejemplo `cosecha:crear`, `labores:validar`).

Rol	Usa	Puede hacer
Administrador	Web	Todo: usuarios, roles, catálogos, parámetros, dispositivos
Gerente	Web	Ver todos los módulos, mapas, pronósticos y reportes
Supervisor	Web y móvil	Validar registros, resolver conflictos, asignar órdenes de trabajo
Técnico de sanidad	Móvil	Plagas, preaviso, trampas, alertas de Fusarium, rutas
Caporal	Móvil	Asistencia y labores de su cuadrilla, enfunde, cosecha
Trabajador	Móvil	Ver sus tareas y su producción; alerta de Fusarium

- La app arma su menú según los permisos del usuario que inició sesión.
- Celulares compartidos: varios usuarios por dispositivo, cambio rápido con PIN o gafete QR.
- Los permisos se sincronizan y se aplican también sin señal.
- Toda creación, edición, validación y borrado queda en **bitacora** con usuario, dispositivo y hora.

Diseño con la guía de marca Kalo

Aplice la *Guía de marca para informes* de Inversiones Kalo (en **docs/marca/**) en la app y el panel. Defina los tokens una sola vez en **packages/ui-tokens** y úselos en móvil (StyleSheet) y web (tema de Tailwind).

```
{
  "marca": { "negro": "#000000", "verde": "#a0d33e", "verde600": "#8bbb31",
"verde700": "#719a24", "verde100": "#eaf6cf", "verde50": "#f5fbe9" },
  "neutros": { "n0": "#ffffff", "n50": "#faf9f7", "n200": "#e2ded7", "n300":
"#c9c4ba", "n500": "#6f6a62", "n700": "#3a3733", "n900": "#141311" },
  "estados": { "exito": "#4f8f2a", "alerta": "#d3a047", "peligro": "#a33323",
"info": "#3d6b8f" },
  "graficas": { "serie1": "#141311", "serie2": "#a0d33e", "serie3":
"#9d978c", "serie4": "#c9c4ba", "cuadrícula": "#e2ded7", "ejes": "#6f6a62" }
}
```

Reglas de la guía:

- Fondo blanco; títulos en #141311 , texto en #3a3733 , texto secundario en #6f6a62 .
- El verde es solo acento: líneas, unidades, botón principal. Nunca rellenos grandes de verde detrás de texto.
- Tipografía Archivo (Google Fonts): títulos en Archivo 800 en MAYÚSCULAS; cuerpo en Archivo 400. Anton solo para portadas o divisores de una o dos palabras.
- Línea negra de 2 pt bajo el encabezado y cada título de sección; las demás divisiones en gris #e2ded7 de 1 pt.
- Esquinas rectas en tarjetas, botones, campos y celdas. Sin sombras, degradados ni texturas.
- Botón principal: fondo verde, texto #111111 . Secundario: borde negro sobre blanco.
- Iconos de línea Lucide (lucide-react-native y lucide-react), del color del texto. Nunca emojis.
- Estados siempre con color y palabra (por ejemplo «BAJO META»); el color solo no transmite significado.
- Tablas: encabezados en mayúsculas pequeñas con espaciado amplio; números alineados a la derecha con cifras tabulares.
- Números: punto para miles y coma para decimales; siempre con unidad y periodo.
- Textos de la interfaz en español formal con «usted».
- Logo: usar kalo-logo.png arriba a la izquierda, mínimo 32 px de alto; nunca redibujarlo ni recolorarlo. Si falta, mostrar «INVERSIONES KALO» en estilo título.

Adaptaciones para uso en campo, compatibles con la guía:

- Botones y áreas táctiles de al menos 56 px de alto, para usar con guantes o manos mojadas.
- Texto de cuerpo mínimo de 16 px y alto contraste para leer bajo el sol.
- Los **colores de cinta** son datos, no colores de marca: se muestran como muestras de color con su nombre escrito al lado.
- Formularios de una pregunta por pantalla cuando sea posible, con opciones grandes en lugar de teclado.

Preparado para el futuro

Diseño para que agregar un módulo, un campo o una finca no requiera reescribir el sistema.

- **Registro de módulos:** cada módulo es una carpeta con su manifiesto (nombre, icono, permisos, rutas, tablas). La app y el panel arman el menú leyendo ese registro.

- **Formularios dinámicos:** el motor de formularios lee `definiciones_formulario` (JSON versionado con campos, tipos, validaciones y lógica condicional) que se descargan al sincronizar. Así se agrega un campo sin publicar una nueva versión. Use este motor para al menos el formulario de muestreo de plagas.
- **Versionado:** cada registro guarda la versión del formulario con que se creó.
- **Feature flags** por finca y por rol para activar módulos de forma gradual.
- **Multi-finca y multi-empresa** desde el diseño (`empresa_id` y `finca_id`).
- **Unidades configurables:** hectáreas o manzanas.
- **Parámetros editables:** umbrales, intervalos de GPS, tamaño de celda de cobertura, factores por semana; nada de eso fijo en el código.
- **Idiomas:** español como base y estructura lista para agregar idiomas mayas.
- **API versionada** (`/v1`) y documentada en OpenAPI, para integrar Habacuscus u otros sistemas.
- **Actualizaciones remotas** con Expo EAS Update.
- **Puntos de extensión** documentados para: servicio de visión (conteo de cintas, sigatoka), imágenes satelitales, nómina, órdenes de trabajo y servidor local en la finca.

Datos de demostración

Cree un script `pnpm seed` que cargue datos ficticios, todos marcados como «DEMO», para que el demo se vea completo.

- 1 empresa (Inversiones Kalo) y 1 finca llamada «Finca Demo».
- 6 lotes con polígonos ficticios ubicados en una zona bananera de Guatemala, con hectáreas y población de 1.600 plantas/ha.
- Un usuario por rol, con PIN conocido documentado en el README.
- 2 cuadrillas con 8 trabajadores ficticios cada una (DPI de prueba, no reales).
- Calendario de 52 semanas del año en curso con colores de cinta de ejemplo y el factor interpolado. Deje claro en la configuración que los colores reales están pendientes de confirmar.
- Catálogo de plagas: sigatoka negra, picudo negro, nematodos, cochinilla, trips y Fusarium R4T (con umbrales de ejemplo editables).
- Catálogo de labores: enfunde, deshoje, deshije, cosecha, apuntalamiento, fertilización.
- 10 trampas con QR imprimibles (generar PDF de códigos).
- 8 semanas de registros simulados de muestreos, enfunde, cosecha, labores y rutas GPS, para que mapas, tablas y pronóstico tengan contenido.
- Un script `pnpm seed:reset` que borra y vuelve a cargar todo.

Entregables y criterios de aceptación

Entregables

- Monorepo con la estructura indicada, que instala con `pnpm install` y levanta con `docker compose up + pnpm dev`.
- App móvil ejecutable en Expo Go o en una build de desarrollo para Android.
- Panel web en `localhost:3000`, API en `localhost:4000` con documentación en `/docs`.
- `README.md` con requisitos, instalación, variables de entorno, usuarios demo y guion del demo.
- Documentos en `docs/`: arquitectura, sincronización y cómo agregar un módulo nuevo.
- Pruebas: cálculo de la ecuación productiva, pronóstico, permisos y sincronización con conflictos.

El demo se considera listo cuando:

- ☐ Un técnico inicia sesión con PIN y, en modo avión, registra un muestreo con foto y GPS.
- ☐ Dos celulares registran datos sin conexión y, al reconectarse, todo llega al servidor sin duplicados.
- ☐ Un conflicto en un registro de cosecha aparece en la bandeja del supervisor.
- ☐ La app detecta el lote por GPS y graba la ruta de un recorrido.
- ☐ El panel muestra los registros nuevos en el mapa, la cobertura del lote y las tablas.
- ☐ El pronóstico semanal de cajas usa el factor de la semana correspondiente.
- ☐ Un caporal solo ve los módulos de su rol.
- ☐ Un campo nuevo agregado a la definición del formulario aparece en la app tras sincronizar, sin nueva versión.
- ☐ La interfaz cumple la guía de marca Kalo (revisión final contra la guía).

Orden de trabajo

Trabaje por etapas. Al terminar cada una, verifique que compila y funciona, haga un commit y resuma lo hecho antes de seguir.

1. **Base:** monorepo, configuración compartida, tokens de diseño Kalo, Docker Compose con Postgres+PostGIS y MinIO.
2. **Datos:** esquema Drizzle, migraciones, paquete `shared` con tipos, Zod y cálculos (ecuación, factor por semana, pronóstico), y seed.
3. **API:** autenticación, permisos, endpoints por módulo, archivos y sincronización pull/push.

4. **App móvil – núcleo:** login con PIN, base WatermelonDB, motor de sincronización, cola de archivos, indicador de estado y menú por rol.
5. **App móvil – módulos:** plagas (con formulario dinámico), trampas, alerta de Fusarium, enfunde, cosecha, labores y asistencia.
6. **GPS:** geocercas, rastreo por tarea, mapas offline y cobertura.
7. **Panel web:** mapa, tablas, pronóstico, validación, alertas, administración y dispositivos.
8. **Cierre:** pruebas, documentación, guion del demo y revisión de toda la interfaz contra la guía de marca Kalo.

Si alguna decisión no está definida en este prompt, elija la opción más simple que no impida crecer, anótela en `docs/decisiones.md` y continúe. Si falta un dato del negocio, déjelo como parámetro configurable y márkelo como pendiente; no lo invente.